



Faculty of Engineering & Technology
Electrical & Computer Engineering Department

COMPUTER ARCHITECTURE - ENCS 4370

Project 2

Multi-Cycle RISC Processor in Verilog

Abstract:

The aim of this project is to design a multicycle RISC processor using Verilog HDL, this processor supports 21 instructions, each of them is with 16-bit size, our processor has 8 16-bit general purpose registers, PC register as well as separate Instruction and Data memories, this processor supports four instruction types: R-Type, I-Type, J-Type and S-Type and it has an ALU unit to perform the computation processes and generate the required flags to be used by the control units in the processor.

Table of Contents

• Abstract	I
• Theory	2
• 3.1. Introduction to RISC Architecture	2
• 3.2. Multi-Cycle Processor Design	2
• 3.3. Design of the Multi-Cycle RISC Processor	2
• Data Path	5
• Cycle-by-Cycle Operation	7
• Control Signals	10
• State Diagram	15
• Testing and Verification	18

Table of Figures

Figure 1: R-type instruction format	2
Figure 2: I-type instruction format	2
Figure 3: J-type instruction format	2
Figure 4: S-type instruction format	3
Figure 5: instruction set	4
Figure 6: Data path schematic	6
Figure 7: ALU Operations and ALU Source	11
Figure 8: branch table flags	12
Figure 9: control signals table	13
Figure 10: state diagram	16

Introduction AND Design Specification:

1. Introduction to RISC Architecture

Reduced Instruction Set Computer (RISC) architecture is a type of microprocessor architecture that utilizes a small, highly optimized set of instructions, rather than a more specialized set of instructions often found in Complex Instruction Set Computer (CISC) architectures. The key characteristics of RISC include:

- **Simplicity:** RISC architectures use a limited number of simple instructions. This simplicity allows for faster execution of each instruction.
- **Uniform Instruction Length:** Instructions are typically of a fixed length, which simplifies the process of fetching and decoding instructions.
- **Load/Store Architecture:** Separate instructions are used for memory access (load/store) and arithmetic/logic operations, which helps in optimizing the execution flow.

2. Multi-Cycle Processor Design

In a multi-cycle processor design, the execution of an instruction is divided into multiple steps, with each step being executed in a different clock cycle. This allows for more efficient use of the processor's resources as different parts of the processor can be used for different purposes in each cycle. The key stages in a multi-cycle processor typically include:

- **Instruction Fetch (IF):** Fetch the instruction from the instruction memory.
- **Instruction Decode/ Register Fetch (ID):** Decode the instruction and read the registers.
- **Execution/ Effective Address Calculation (EX):** Perform the necessary arithmetic or logic operation or calculate the effective address.
- **Memory Access (MEM):** Access data memory if the instruction involves a load or store operation.
- **Write Back (WB):** Write the results back to the register file.

In each clock cycle, only a part of the instruction is executed, and different instructions may take a different number of cycles to complete.

3. Design of the Multi-Cycle RISC Processor

The multi-cycle RISC processor designed in this project is based on a 16-bit instruction and word size architecture with eight general-purpose registers. The processor supports a variety of instruction types, including R-type, I-type, J-type, and S-type instructions, each serving different purposes.

4. Processor Specifications:

1. **Instruction size and word size:** 16 bits
2. **Registers:**
 - 8 general-purpose registers (R0-R7), R0 is hardwired to zero.
 - 16-bit program counter (PC)
3. **Instruction types:**
 - R-Type: The R-type instructions in our processor like: ADD, SUB and AND have the following format

R-Type (Register Type)

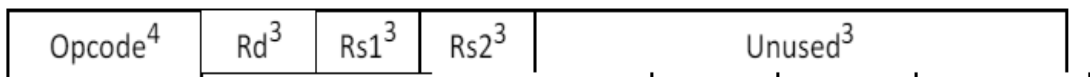


Figure 1: R-type instruction format

- I-Type: The I-type instructions in our processor like: ADDI, ANDI, Branch, LW, SW, LBs and LBU have the following format:

I-Type (Immediate Type)

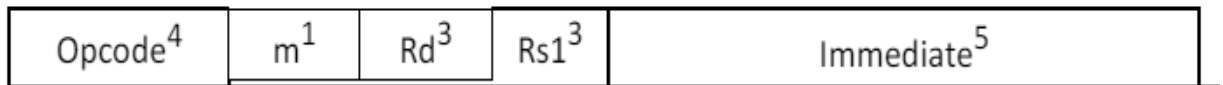


Figure 2: I-type instruction format

- J-Type: The J-type instructions in our processor like: JMP, CALL and RET have the following format

J-Type (Jump Type)

This type includes the following instruction formats. The opcode is used to distinguish each instruction

```
jmp    L # Unconditional jump to the target L.
call   F # Call the function F. F is a label.
        # The return address is pushed on r15.
```

The target address is calculated by concatenating the most significant 7-bit of the current PC with the 12-bit offset after multiplying offset by 2.

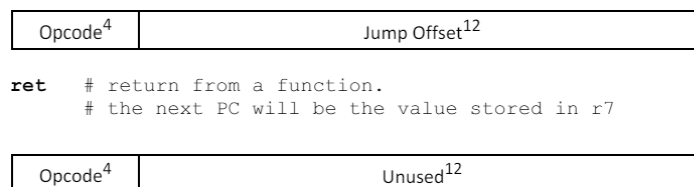


Figure 3: J-type instruction format

- S-Type: The S-type instructions in our processor like: SV have the following format

S-Type (Store)

Opcode ⁴	Rs ³	Immediate ⁸
---------------------	-----------------	------------------------

Figure 4:S-type instruction format

4. List of supported instructions:

No.	Instr	Format	Meaning	Opcode Value	m
1	AND	R-Type	Reg(Rd) = Reg(Rs1) & Reg(Rs2)	0000	
2	ADD	R-Type	Reg(Rd) = Reg(Rs1) + Reg(Rs2)	0001	
3	SUB	R-Type	Reg(Rd) = Reg(Rs1) - Reg(Rs2)	0010	
4	ADDI	I-Type	Reg(Rd) = Reg(Rs1) + Imm	0011	
5	ANDI	I-Type	Reg(Rd) = Reg(Rs1) + Imm	0100	
6	LW	I-Type	Reg(Rd) = Mem(Reg(Rs1) + Imm)	0101	
7	LBU	I-Type	Reg(Rd) = Mem(Reg(Rs1) + Imm)	0110	0
8	LBS	I-Type	Reg(Rd) = Mem(Reg(Rs1) + Imm)	0110	1
9	SW	I-Type	Mem(Reg(Rs1) + Imm) = Reg(Rd)	0111	
10	BGT	I-Type	if (Reg(Rd) > Reg(Rs1)) Next PC = PC + sign_extended (Imm) else PC = PC + 2	1000	0
11	BGTZ	I-Type	if (Reg(Rd) > Reg(0)) Next PC = PC + sign_extended (Imm) else PC = PC + 2	1000	1
12	BLT	I-Type	if (Reg(Rd) < Reg(Rs1)) Next PC = PC + sign_extended (Imm) else PC = PC + 2	1001	0
13	BLTZ	I-Type	if (Reg(Rd) < Reg(R0)) Next PC = PC + sign_extended (Imm) else PC = PC + 2	1001	1
14	BEQ	I-Type	if (Reg(Rd) == Reg(Rs1)) Next PC = PC + sign_extended (Imm) else PC = PC + 2	1010	0
15	BEQZ	I-Type	if (Reg(Rd) == Reg(R0)) Next PC = PC + sign_extended (Imm) else PC = PC + 2	1010	1
16	BNE	I-Type	if (Reg(Rd) != Reg(Rs1)) Next PC = PC + sign_extended (Imm)	1011	0
			else PC = PC + 2		
17	BNEZ	I-Type	if (Reg(Rd) != Reg(Rs1)) Next PC = PC + sign_extended (Imm) else PC = PC + 2	1011	1
18	JMP	J-Type	Next PC = {PC[15:10], Immediate}	1100	
19	CALL	J-Type	Next PC = {PC[15:10], Immediate} PC + 4 is saved on r15	1101	
20	RET	J-Type	Next PC = r7	1110	
21	Sv	S-Type	M[rs] = imm	1111	

Figure 5: instruction set

4. Memory: our design will have Separate data and instruction memories, byte addressable, little endian.

5. ALU: In our processor we will have an ALU that calculates the arithmetic and logic operations as well as generating the required signals for branch outcomes (zero, carry, overflow, etc.).

5. Data Path:

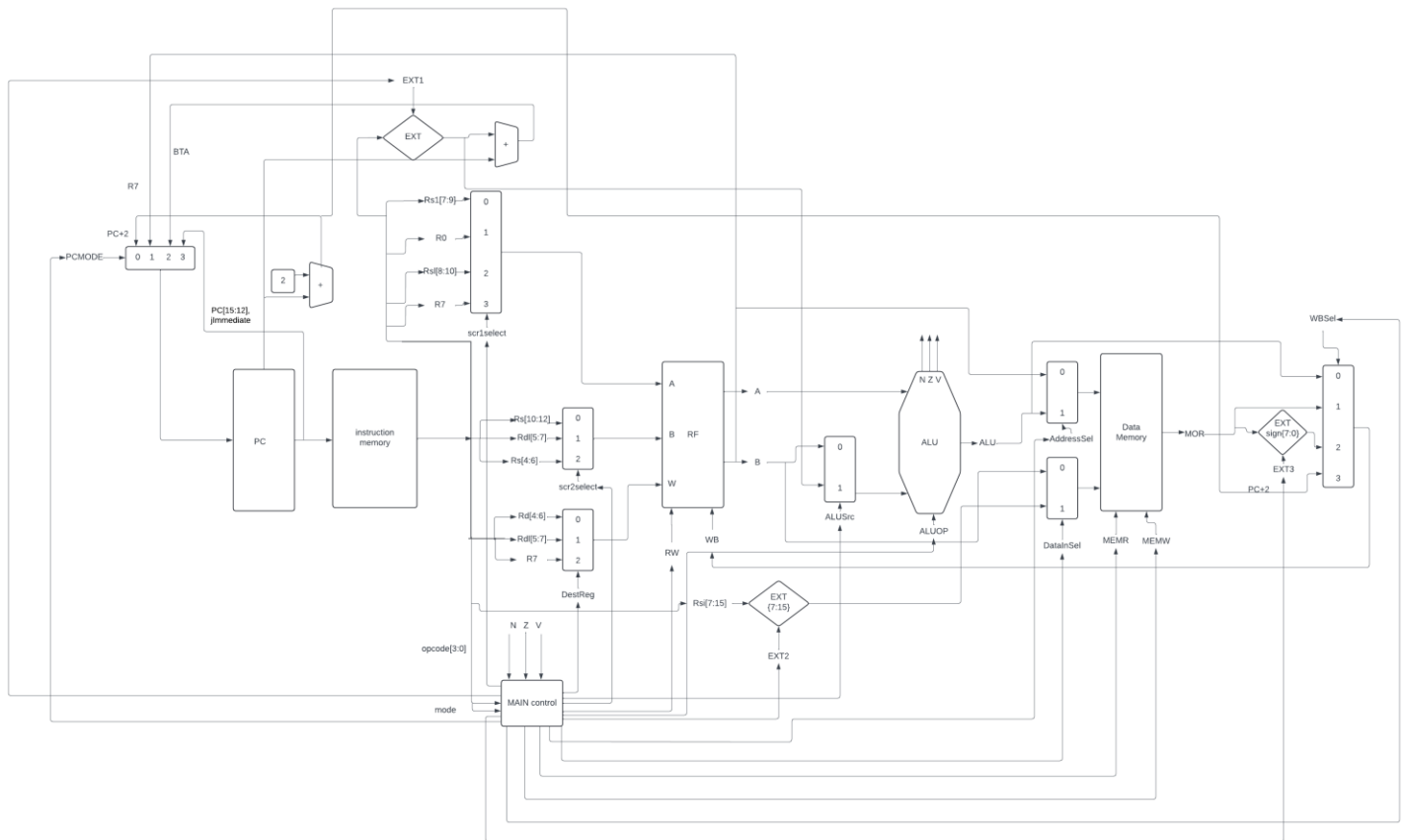


Figure 6:Data path schematic

Instruction Fetch (IF)

1. Program Counter (PC):

- The PC holds the address of the next instruction to be executed. It is updated every cycle to point to the new instruction.
- Incrementing the PC: Typically, the PC is incremented by the size of the instruction using an adder.

2. Instruction Memory:

- Fetches the instruction located at the address specified by the PC.
- The fetched instruction is then placed into the Instruction Register (IR).

Instruction Decode (ID)

1. Instruction Register (IR):

- Holds the fetched instruction to be decoded and processed in subsequent stages.

2. Control Unit:

- Decodes the instruction opcode to generate control signals for the entire Datapath.
- These signals include:
 - Register read/write signals.
 - ALU operation signals.
 - Memory read/write signals.
 - Multiplexer control signals.
- Decodes the ALU operation specified by the instruction (e.g., ADD, SUB, AND).
- Generates control signals to perform the correct ALU operation.

3. Register File (RF):

- Contains a set of registers used for storing operands and intermediate results.
- Source registers specified by the instruction are read into two temporary registers (often labeled as A and B).

Execution (EX)

1. Arithmetic Logic Unit (ALU):

- Performs arithmetic and logical operations required by the instruction.
- Inputs to the ALU can come from:
 - Registers A and B.
 - Immediate values (often sign-extended before being used).
 - The current PC value for branch calculations.

Memory Access (MEM)

1. Data Memory:

- Accessed when the instruction involves load/store operations.
- The address for memory access is computed by the ALU.
- The control unit sends read/write signals to access data memory.

Write Back (WB)

1. Register Write:

- Writes the result of the ALU operation or the data fetched from memory back to the destination register in the register file.
- The destination register is specified by the instruction.

2. Multiplexers (MUX):

- Used to select between different data sources for various operations:
 - Choosing between ALU result and memory data for the write-back stage.
 - Selecting between different sources for ALU operands.

Cycle-by-Cycle Operation

Cycle 1: Instruction Fetch (IF)

- PC provides the address to Instruction Memory.
- Instruction Memory fetches the instruction.
- PC is incremented to point to the next instruction.

Cycle 2: Instruction Decode (ID)

- IR holds the fetched instruction.
- Control Unit decodes the instruction and sets control signals.
- Register File reads source registers into A and B.

Cycle 3: Execution (EX)

- ALU performs the required operation using inputs from A, B, or immediate values.
- ALU Control Unit sets the operation mode for the ALU.

Cycle 4: Memory Access (MEM)

- Data Memory is accessed if needed (for load/store instructions).
- The address for memory access is computed by the ALU.

Cycle 5: Write Back (WB)

- Register File writes the result of the ALU operation or memory data back to the destination register.
- Multiplexers select the appropriate data source for the write-back stage.

Example Instruction Flow

Let's walk through an example of a typical instruction, such as an `ADD` instruction:

1. Fetch: The instruction is fetched from memory using the address in the PC and placed in the IR.
2. Decode: The instruction is decoded, identifying the operation (ADD) and the source/destination registers.
3. Execute: The ALU performs the addition operation on the operands read from the register file.
4. Memory Access: (Not needed for ADD, but would be for load/store instructions).
5. Write Back: The result of the addition is written back to the destination register in the register file.

Each stage operates on a different instruction during each cycle, allowing the Datapath to handle multiple instructions concurrently in different stages, thus optimizing CPU performance and resource utilization. This multi-cycle approach ensures a balance between hardware complexity and execution efficiency, making it a widely used design in CPU architectures.

Control Signals:

This table defines which ALU operation and ALU source to use for each instruction.

OP	ALU_Op	ALU_Src
AND	And	00
ADD	Add	01
SUB	Sub	10
ADDI	Add	01
ANDI	And	01
LW	Add	01
LBU	Add	01
LBS	Add	01
SW	Add	01
BGT	Sub	10
BGTZ	Sub	10
BLT	Sub	10
BLTZ	Sub	10
BEQ	Sub	10
BEQZ	Sub	10
BNE	Sub	10
BNEZ	Sub	10
JMP	X	X
CALL	X	X
RET	X	X
SV	X	X

Table 1: ALU Operations and ALU Source

-OP: The mnemonic for the instruction.

- ALU_Op: The operation the ALU should perform.

- And: Perform a bitwise AND operation.

- Add: Perform an addition.

- Sub: Perform a subtraction.

- XX: No ALU operation (used for jump and call instructions which don't involve the ALU).

- ALU_Src: The source for the ALU operation.

- 00: Use both register operands.

- 01: Use one register and an immediate value.

- 10: Used for branch instructions comparing register values.

This table specifies the conditions under which the Program Counter (PC) should be updated and the source of the new PC value.

OP	Flgs	PC/Src
BGT & BGTZ	(Z == 1) (N != V)	10
BLT & BLTZ	(Z == 1) (N == V)	10
BEQ & BEQZ	(Z == 1)	10
BNE & BNEZ	(Z == 0)	10
JMP & CALL	X	11
RET	X	01
else	X	00

Table 2: Branch Operations, Flags, and PC Source

- OP: The branch operation.
- Flgs: The condition flags used to determine if the branch should be taken.
 - Z: Zero flag.
 - N: Negative flag.
 - V: Overflow flag.
- PC/Src: The source for the new Program Counter value.
 - 10: Use the address computed by the ALU (usually for branch instructions).
 - 11: Use the immediate value (for jump and call instructions).
 - 01: Use the return address (for the RET instruction).
 - 00: No change to the PC (default case for instructions that do not change the PC).

This table provides detailed control signals for various instructions, indicating how registers, ALU, memory, and write-back operations are configured.

OP	mode	Reg1Sel	Reg2Sel	DestRegSel	Ext1	Ext2	Ext3	ALUSrc	AddressSel1	DataInSel	WBSEL	MEMR	MEMW	REGW
R-Type	X	00	00	00	X	X	X	0	X	X	00	0	0	1
ADDI	X	10	X	01	1	X	X	1	X	X	00	0	0	1
ANDI	X	10	X	01	0	X	X	1	X	X	00	0	0	1
LW	X	10	X	01	1	X	X	1	1	X	01	1	0	1
LHu	0	10	X	01	1	X	1	1	1	X	10	1	0	1
LBz	1	10	X	01	1	X	0	1	1	X	10	1	0	1
SW	X	10	01	X	1	X	X	1	1	0	X	0	1	0
BGT	0	10	01	X	1	X	X	0	X	X	X	0	0	0
BGTZ	1	01	01	X	1	X	X	0	X	X	X	0	0	0
BLT	0	10	01	X	1	X	X	0	X	X	X	0	0	0
BLTZ	1	01	01	X	1	X	X	0	X	X	X	0	0	0
BEQ	0	10	01	X	1	X	X	0	X	X	X	0	0	0
BEQZ	1	01	01	X	1	X	X	0	X	X	X	0	0	0
BNE	0	10	01	X	1	X	X	0	X	X	X	0	0	0
BNEZ	1	01	01	X	1	X	X	0	X	X	X	0	0	0
JMP	X	X	X	X	X	X	X	X	X	X	X	0	0	0
CALL	X	X	X	10	X	X	X	X	X	X	11	0	0	1
RET	X	11	X	X	X	X	X	X	X	X	X	0	0	0
sv	X	X	10	X	X	1	X	X	0	1	X	0	1	0

Table 3: Comprehensive Control Signals

- **OP:** instruction.
- **mode:** A specific mode or condition for the instruction.
- **Reg1Sel:** Selection signal for the first source register.

- 00: Selects a specific register.

- 10: Indicates the use of register 1.

→ Boolean expression:

$$\text{Reg1Sel}[0] = \overline{(\text{R-Type} + \text{BGTZ} + \text{BLTZ} + \text{BEQZ} + \text{BNEZ})}$$

$$\text{Reg2Sel}[1] = \overline{(\text{R-Type} + \text{Branch} + \text{SW})}$$

- **Reg2Sel:** Selection signal for the second source register.

- 00: Selects a specific register.

- 01: Indicates the use of register 2.

→ Boolean expression:

$$\text{Reg2Sel}[0] = \overline{(\text{R-Type} + \text{Branch})}$$

$$\text{Reg2Sel}[1] = \overline{(\text{R-Type} + \text{SV})}$$

- **DestRegSel:** Selection signal for the destination register.

- 00: Selects a specific register.

- 01: Indicates the use of a different register.

→ Boolean expression:

$$\text{DestRegSel} [0] = \overline{(\text{R-Type} + \text{ADDI} + \text{ANDI} + \text{LBU} + \text{LBS})}$$

$$\text{DestRegSel} [1] = \overline{(\text{R-Type} + \text{CALL})}$$

- **Ext1:** Extension or immediate value.

- 1: Use the immediate value.

- x: Don't care (not used).

→ Boolean expression:

$$\text{Ext1} = \overline{(\text{ADDI})}$$

- **Ext2:** Another extension for immediate value used with S-type instructions.

→ Boolean expression:

$$\text{Ext2} = \text{SV}$$

- **Ext3:** Another extension for immediate value used with LBU and LBS instructions.

→ Boolean expression:

$$\text{Ext3} = \text{LBU} \ \& \ !\text{mode}$$

- **ALUSrc:** ALU source.

- 0: Use register values.

- 1: Use an immediate value.

→ Boolean expression:

$$\text{ALUSrc} = (\text{ADDI} + \text{ANDI} + \text{LW} + \text{LBU} + \text{LBS} + \text{SW})$$

- **AddressSel:** Address selection for memory operations.

- 1: Use the computed address.

- x: Don't care (not used).

→ Boolean expression:

$$\text{AddressSel} = \overline{(\text{SV})}$$

DataInSel: Selection signal for input data.

-x: Don't care (not used).

→ Boolean expression:

$$\text{DataInSel} = (\text{SV})$$

- **WBSEL**: Write-back selection.

- 00: Default write-back.

- 01: Write-back from memory.

- 10: Write-back from a different source.

- 11: Write-back from another different source.

→ Boolean expression:

$$\text{WBSEL}[0] = (\text{LBu} + \text{JMP})$$

$$\text{WBSEL}[1] = (\text{LW} + \text{JMP})$$

- **MEMR**: Memory read signal.

- 0: No memory read.

- 1: Perform a memory read.

→ Boolean expression:

$$\text{MEMR} = (\text{LBu} + \text{LW})$$

- **MEMW**: Memory write signal.

- 0: No memory write.

- 1: Perform a memory write.

→ Boolean expression:

$$\text{MEMW} = (\text{SW} + \text{SV})$$

- **REGW**: Register write signal.

- 0: No register write.

- 1: Perform a register write.

→ Boolean expression:

$$\text{REGW} = (\text{R-Type} + \text{ADDI} + \text{ANDI} + \text{LW} + \text{LBu} + \text{CALL})$$

State Diagram:

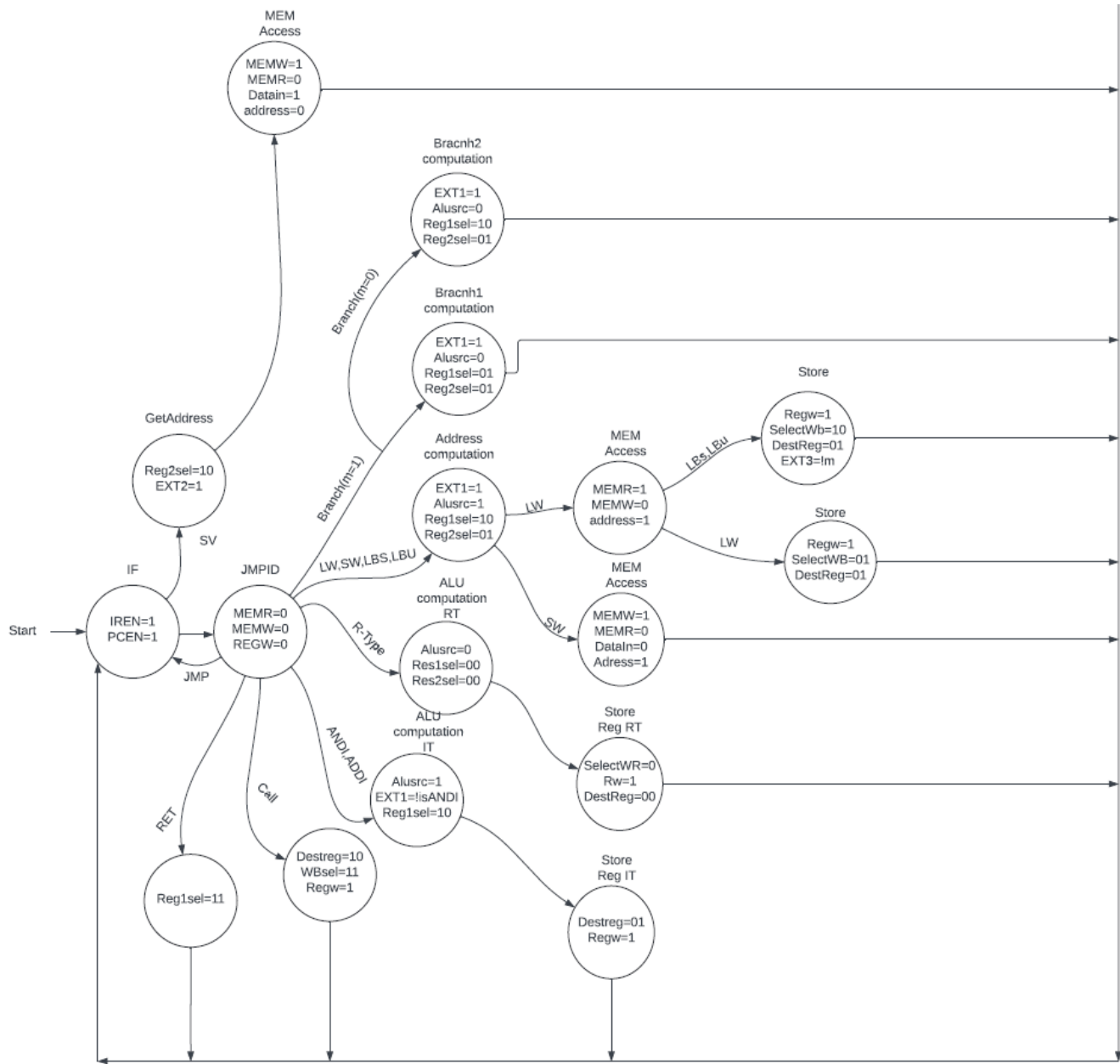


Figure 7:state diagram

This state diagram represents the detailed control signals and state transitions for different types of instructions in a processor, including fetch, decode, execute, and memory access stages. The diagram begins at the **Start** state, where initial control signals are set to prepare for instruction fetching:

- Start State:

- IREN=1, PCEN=1: Initialize Instruction Register Enable and Program Counter Enable.
- MEMR=0, MEMW=0, REGW=0: No memory read/write or register write operations.

Instruction Fetch (IF):

- In this state, the instruction is fetched from memory. The Program Counter (PC) points to the address of the instruction.

Jump (JMP):

- Reg2sel=10, EXT2=1: Selects the second register and extends the immediate value.

Get Address:

- MEMW=1, MEMR=0, Datain=1: Writes data into memory but doesn't read from memory. The data input is enabled.

Save (SV):

- MEM Access: Performs memory access operations, with the first register selected as Reg1sel=11.

Return (RET):

- Destreg=10, WBsel=11, Regw=1: Sets the destination register, selects the write-back source, and enables register writing.

Call:

- Alusrc=1, EXT1=!isANDI, Reg1sel=10: The ALU source is set to immediate, extended value is checked, and the first register is selected.

ANDI, ADDI:

- Alusrc=0, Res1sel=00, Res2sel=00: The ALU source is set to register values, with both result selectors set to default.

Load/Store Instructions (LW, SW, LBS, LBU):

- ALU Computation: Executes ALU operations for address computation.
- EXT1=1, Alusrc=1, Reg1sel=10, Reg2sel=01: Sets the extended value, ALU source, and both register selectors.

R-Type Instructions (RT):

- ALU Computation: Executes ALU operations for the result.
- EXT1=1, Alusrc=0, Reg1sel=01, Reg2sel=01: Configures ALU inputs for register values.

Branch Instructions:

- Branch (m=1):
 - Branch1 Computation: Sets the branch computation parameters.
- Branch (m=0):
 - Branch2 Computation: Configures the branch computation differently.

Memory Access for Load (LW):

- MEMR=1, MEMW=0, address=1: Reads from memory at the computed address.

Memory Access for Store (SW):

- MEMW=1, MEMR=0, DataIn=0, Address=1: Writes data to memory at the computed address.

Register Write Back:

- SelectWR=0, Rw=1, DestReg=00: Sets the write-back signal to register.

- Store Reg RT:

- Destreg=01, Regw=1: Configures the destination register and enables writing.

- Store Reg IT:

- Regw=1, SelectWb=10, DestReg=01: Configures write-back source and destination register.

- Store Load Byte/Unsigned (LBs, LBU):

- Regw=1, SelectWB=01, DestReg=01: Configures write-back for load byte operations.

This detailed explanation covers the various states and transitions within the state diagram, outlining the control signals and configurations needed for different processor operations. Each state is critical for managing the flow of instructions, handling memory operations, and updating registers accurately.

Testing and verification:

Test 1:

In this test we wrote a code that tests the functionality of ADDI, SW and LW instructions.

→ CODE:

```
39 ; First code - Testing Arithmetic & lw/sw
40 ;;;;;;;;;;;;;;;;;;;;;;;;;;;
41 addi r1, r0, 5
42 addi r2, r0, 255
43 sw r2, r5, 0x1
44 lw r7, r5, 0x1
45 ;;;;;;;;;;;;;;;;;;;;;;;;;;;
```

→ RESULT:

The screenshot displays a Verilog simulation environment with two main windows: 'testbench.av' and 'design.av'. The 'testbench.av' window contains the following code:

```
1 // Code your testbench here
2 // or browse Examples
3 `timescale 1ns/1ps
4
5 module full_cpu_tb;
6
7     reg reset;
8     reg clk;
9     wire [15:0] data_out;
10
11     initial begin
12
13         clk = 0 ;
14         forever #5 clk = ~clk ;
15     end
16 endmodule
```

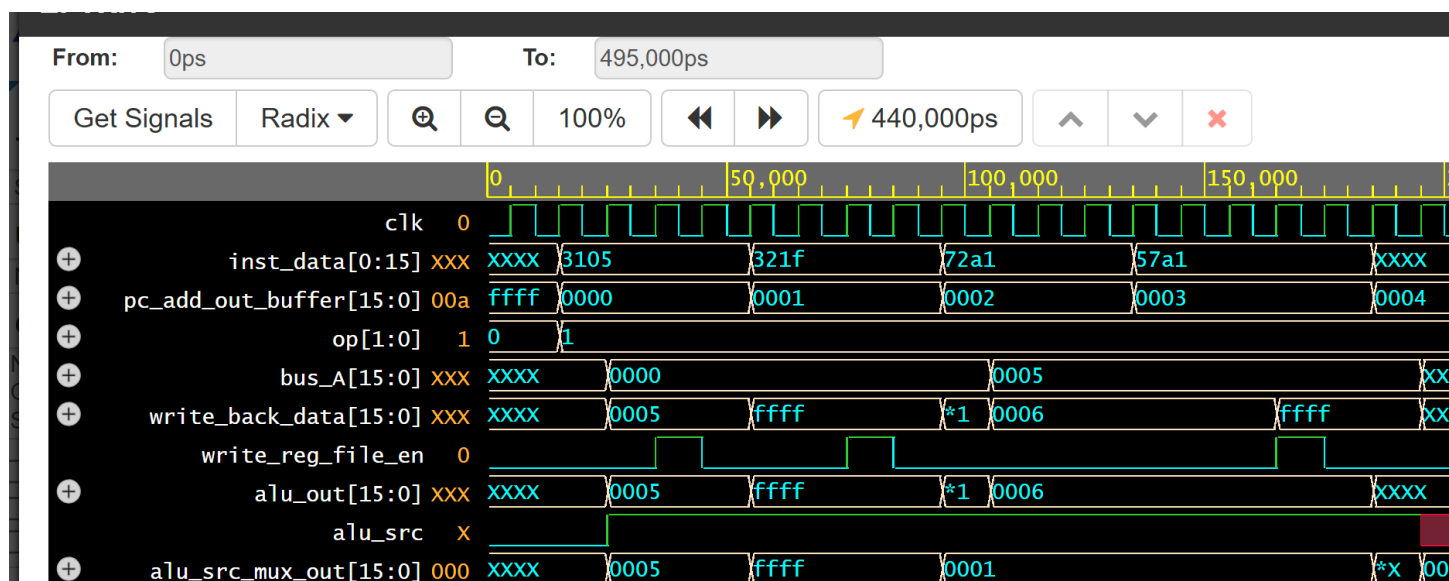
The 'design.av' window shows the 'instructions.dat' file with the following content:

```
1 3105
2 321f
3 72a1
4 57a1
5
```

Below the code windows, the simulation results are displayed. The 'WRITE_BACK Time' is 495. The 'Time :' is 495. The register values are: R0 : 0, R1 : 5, R2 : -1, R3 : 3, R4 : 4, R5 : 5, R6 : 6, R7 : -1. The memory unit read address is x, data_out is x, and Time is 495. The simulation is completed, and the 'Done' button is highlighted.

WRITE_BACK Time 495
Time : 495
R0 : 0, R1 : 5, R2 : -1, R3 : 3, R4 : 4, R5 : 5, R6 : 6, R7 : -1
memory unit read address x data_out x , Time : 495
\$finish called from file "testbench.sv", line 36.
\$finish at simulation time 500000
V C S S i m u l a t i o n R e p o r t
Time: 500000 ps
CPU Time: 0.430 seconds; Data structure size: 0.0Mb
Sat Jun 22 06:49:22 2024
Done

→ **WAVEFORM:**



Test 2:

In this test we wrote a code that get the maximum value between two registers, this test tested the branch and J-type instructions.

→ CODE:

```
34 ; Third code- Max Example
35 ;;;;;;;;;;;;;;;;;;;;;;;;;;
36 addi r1, r0, 5
37 addi r2, r0, 10
38 call max
39 addi r7, r0, 10
40 jmp goout
41
42 max:
43     bgt r1, r2, else
44     add r3, r0, r2
45     jmp endd
46     else:
47         add r3, r0, r1;
48 endd:
49     ret
50 goout:
51 ,
```

→ RESULT:

The screenshot shows the EDA Playground interface. The main editor displays a Verilog code snippet for a CPU module. The right sidebar shows the 'Instructions.dat' file with a list of memory addresses and values. The bottom status bar shows the simulation time at 495 and the output of the 'memory unit read' command.

Verilog Code (testbench.v):

```
// Instantiate the full_cpu module
full_cpu dut (
    .reset(reset),
    .clk(clk),
    .data_out(data_out)
);

// Clock generation
initial begin
    reset = 1'b1;
    #7
    reset = 0 ;
end

// Finish the simulation after a certain time
initial begin
    #500 $finish;
end
endmodule
```

Instructions.dat:

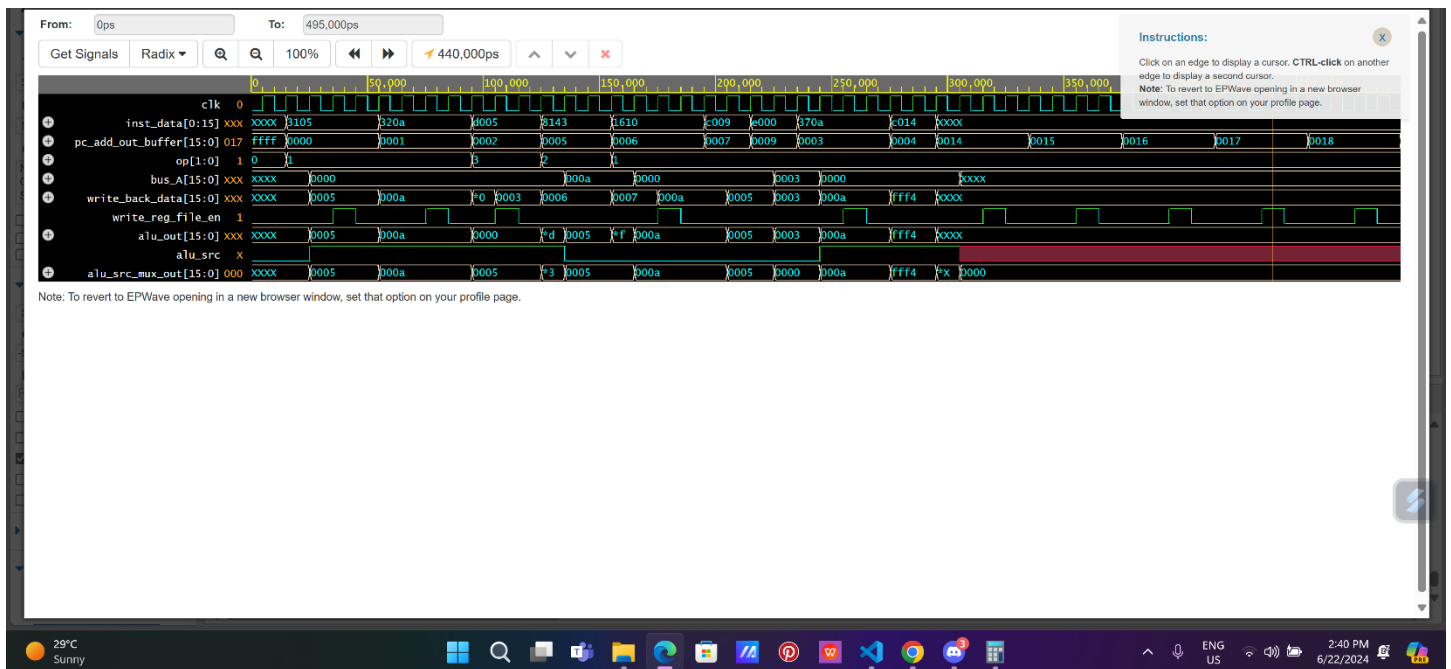
Address	Value
3105	
320a	
4005	
370a	
c014	
8143	
1610	
c009	
1608	
e000	

Simulation Output:

```
Time : 495
R0 : 0, R1 : 5, R2 : 10, R3 : 10, R4 : 4, R5 : 5, R6 : 6, R7 : 10

memory unit read address x data_out x , Time : 495
```

→ WAVEFORM:



Test 3:

In this test we wrote a code that get the minimum value between two registers, this test tested the branch and J-type instructions.

→ CODE:

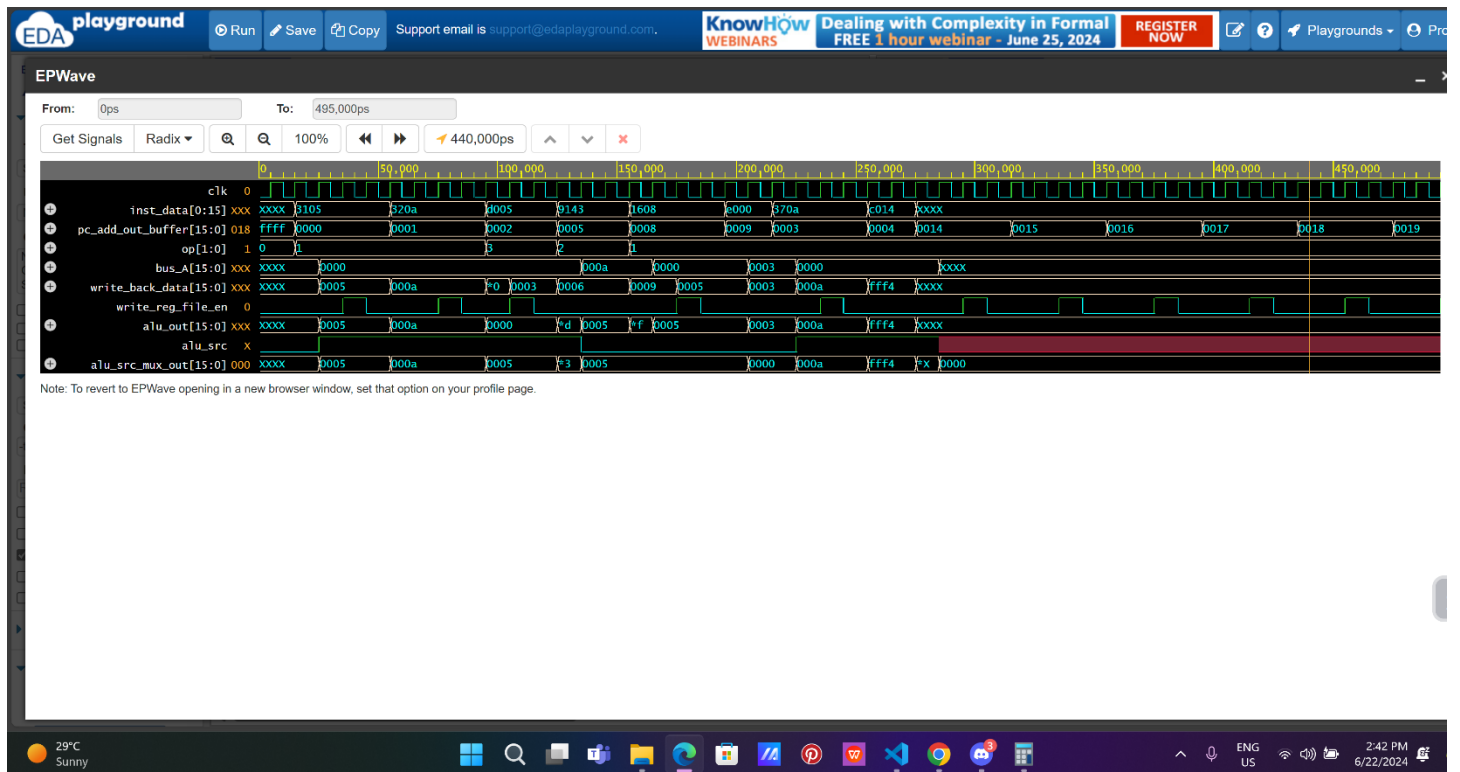
```
60 ; Fourth code- Min Example
61 ;;;;;;;;;;;;;;;;;;;;;;;;;;;
62 addi r1, r0, 5
63 addi r2, r0, 10
64 call min
65 addi r7, r0, 10
66 jmp goout
67
68 min:
69     blt r1, r2, else
70     add r3, r0, r2
71     jmp endd
72 else:
73     add r3, r0, r1
74     endd:
75     ret
76 goout:
```

→ RESULT:

The screenshot displays the EDA Playground web interface. The main editor shows a Verilog testbench for a module named 'full_cpu'. The testbench includes a clock generation section and a simulation finish section. The simulation results are shown in the bottom right, indicating that the simulation completed successfully at time 495. The results table shows the values of registers R0 through R7 at the end of the simulation.

Register	Value
R0	0
R1	5
R2	10
R3	5
R4	4
R5	5
R6	6
R7	10

→ WAVEFORM:



Test 4:

In this test we wrote a code that tests the functionality of BGTZ instruction

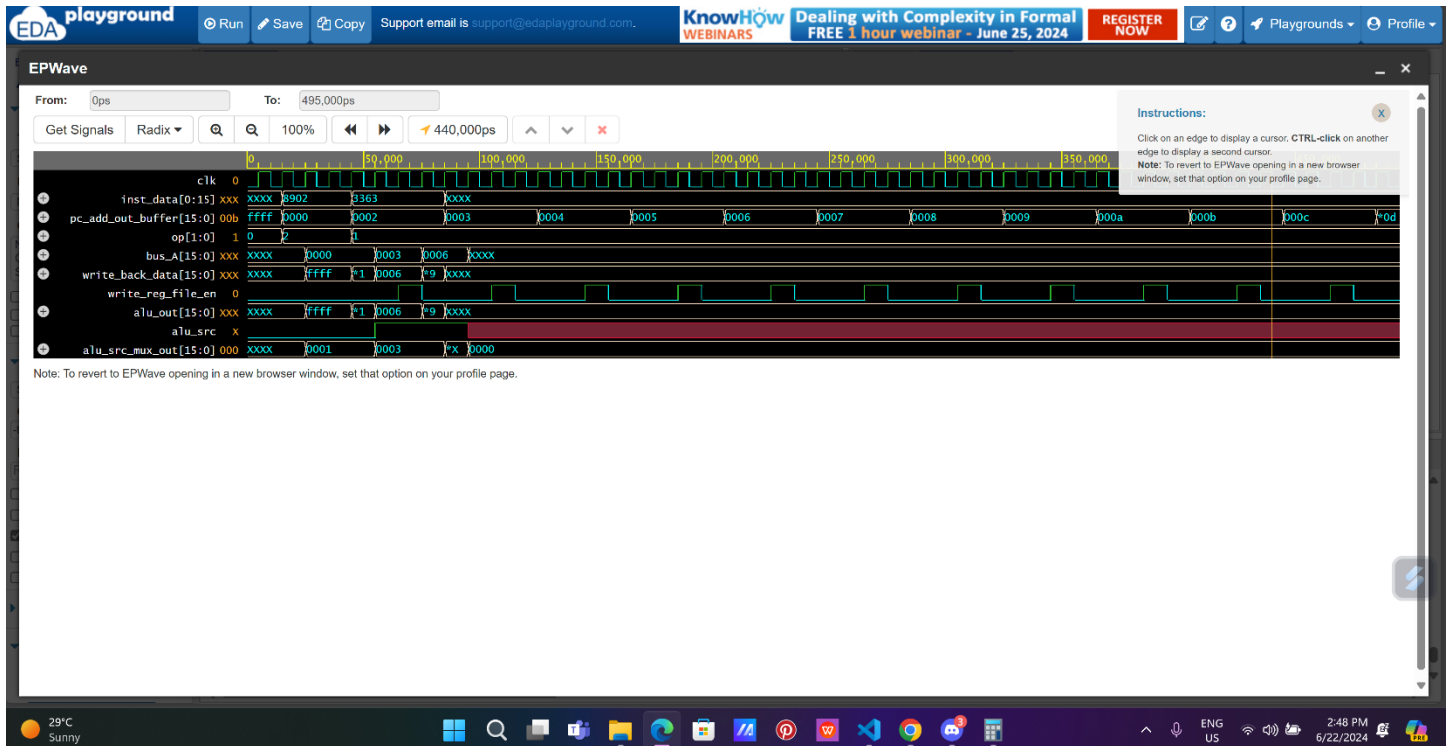
→ CODE:

```
126 bgtz r1, r0, greaterthan
127 jmp finish
128 greaterthan:
129 addi r3, r3, 0x3
130 finish:
131
```

→ RESULT:

The screenshot displays the DOULOS IDE interface during a simulation. The left sidebar shows the 'Languages & Libraries' panel with 'Testbench + Design' selected. The main editor shows the testbench code for 'testbench.av'. The simulation results are displayed in the 'Log' window, showing the register values at time 495: R0: 0, R1: 1, R2: 2, R3: 6, R4: 4, R5: 5, R6: 6, R7: 7. The EPWave window shows a memory unit read at address x, data_out x, at time 495.

→ WAVEFORM:



Test 5:

In this test we wrote a code that tests the functionality of BLTZ instruction

→ CODE:

```
133 addi r1, r1, -2
134 bltz r1, r0, lessthan
135 jmp finish
136 lessthan:
137     addi r3, r3, 0x4
138 finish:
```

→ RESULT:

The screenshot displays the EDA Playground web interface. The top navigation bar includes the EDA Playground logo, a 'Run' button, and a 'Save' button. A banner for 'KnowHow WEBINARS' is visible, along with a 'REGISTER NOW' button. The left sidebar contains a 'Languages & Libraries' section with 'Testbench + Design' and 'Tools & Simulators' tabs. The main workspace is divided into two panels: 'testbench.av' and 'design.av'. The 'testbench.av' panel contains the following Verilog code:

```
// Code your testbench here
// or browse Examples
timescale 1ns/1ps

module full_cpu_tb;

    initial begin
        $dumpfile("dump.vcd");
        $dumpvars;
    end

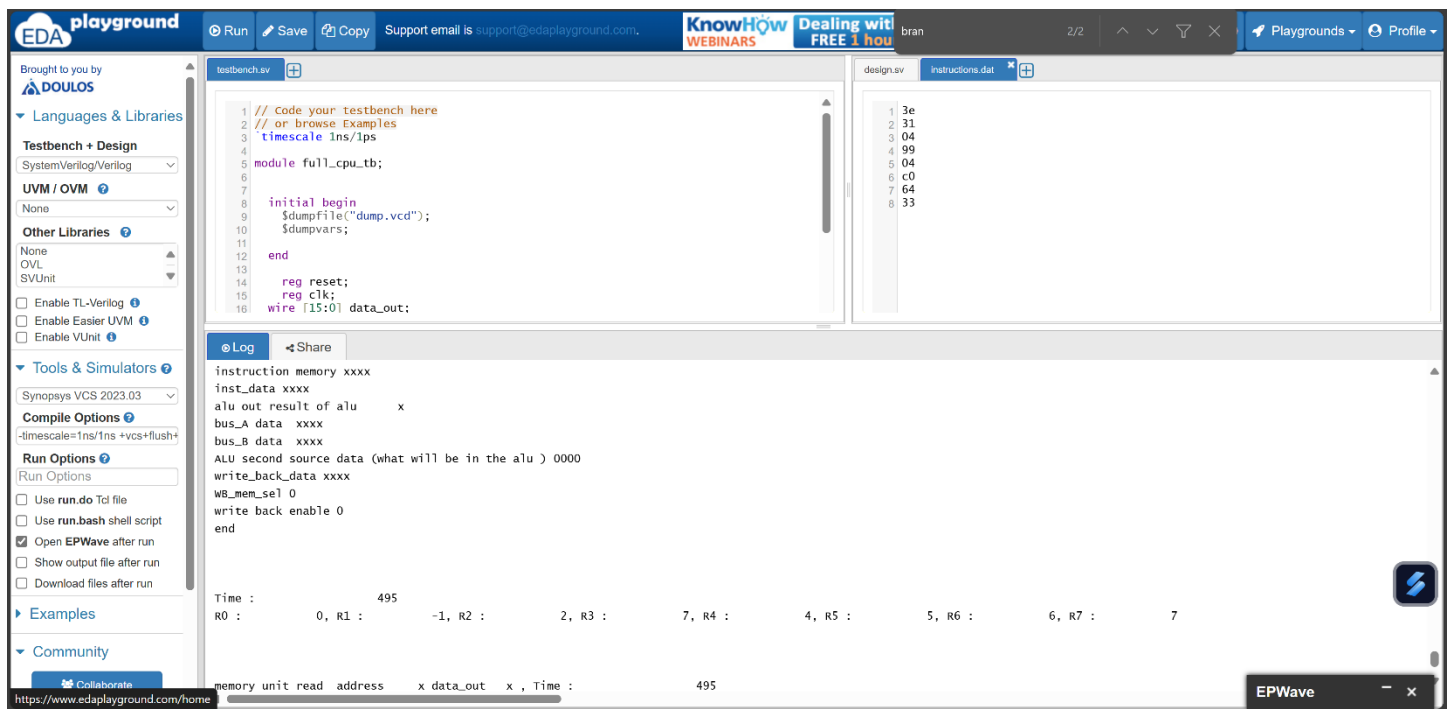
    reg reset;
    reg clk;
    wire [15:0] data_out;

    // ... (rest of the testbench code) ...
```

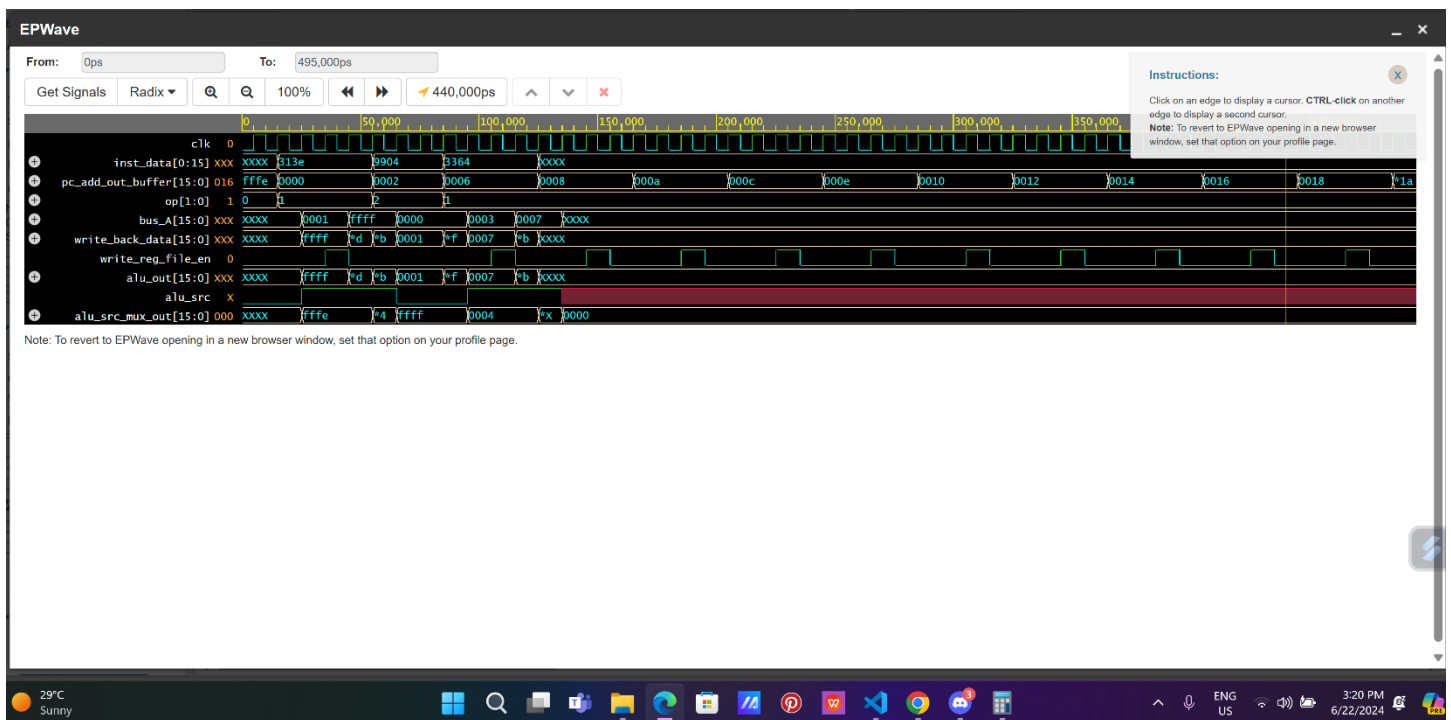
The 'design.av' panel shows the compiled code for the BLTZ instruction. The simulation results are displayed in the bottom panel, showing the state of the CPU registers (R0, R1, R2, R3, R4, R5, R6, R7) and the data output (data_out) over time. The results show that the BLTZ instruction correctly branches to the 'lessthan' label when the condition is met.

→ Test5 using byte addressable memory

→ **RESULT:**



→ WAVEFORM:



Test 6:

In this test we wrote a code that tests the functionality of LBU and LBS instructions

→ CODE:

```
114 add r3, r2, r6|
115 sw r3, r5, 0x1
116 lbu r7, r5, 0x1
117 lbs r6, r5, 0x1
118
```

→ RESULT:

The screenshot displays the EDA Playground web interface. The top navigation bar includes the EDA Playground logo, a 'Run' button, a 'Save' button, and a 'Copy' button. Below the navigation bar, the left sidebar contains a 'Languages & Libraries' section with a 'Testbench + Design' dropdown set to 'SystemVerilog/Verilog'. The main area is split into two panels: 'testbench.sv' on the left and 'design.sv' on the right. The 'testbench.sv' panel shows a Verilog testbench for a CPU module, including a clock generation loop and a call to the 'full_cpu' module. The 'design.sv' panel shows a Verilog module for a CPU, including a register array and a read operation. Below the code panels, the 'Log' tab is active, displaying simulation results. The results show the state of registers R0 through R7 at time 495, with R0 at 0, R1 at 1, R2 at 256, R3 at 392, R4 at 4, R5 at 5, R6 at 136, and R7 at -120. The bottom status bar shows the temperature as 73°F, the time as 4:49 PM, and the date as 6/22/2024.

```
12 end
13
14 reg reset;
15 reg clk;
16 wire [15:0] data_out;
17
18 initial begin
19
20   clk = 0 ;
21   forever #5 clk = ~clk ;
22 end
23
24 // Instantiate the full_cpu module
25 full_cpu dut (
26   .reset(reset),
27   .clk(clk),
28   .data_out(data_out)
29 );
```

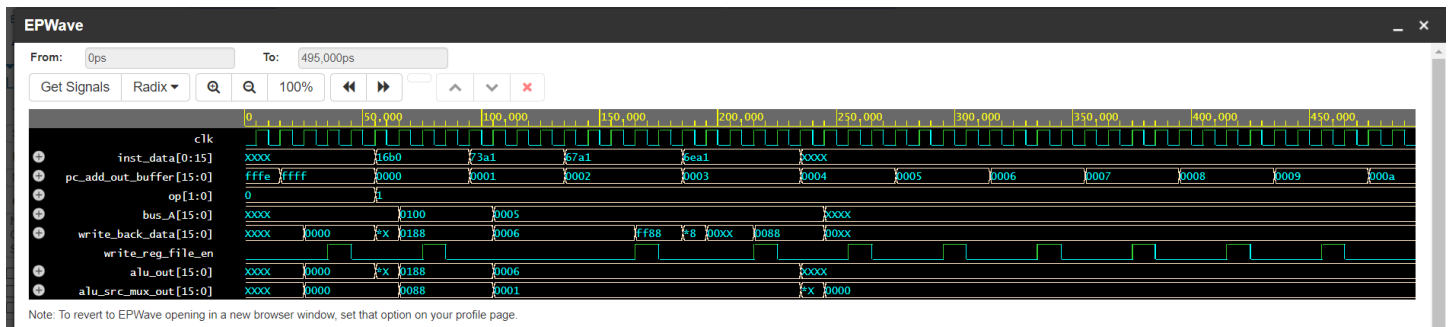
```
789 );
790
791 // Register array, 7 registers of 16 bits each
792 reg signed [15:0] registers [0:7];
793
794 // Initialize r0 to 0
795 initial begin
796   registers[0] = 16'h0000;
797   registers[1] = 16'h0001;
798   registers[2] = 16'h0100;
799   registers[3] = 16'h0003;
800   registers[4] = 16'h0004;
801   registers[5] = 16'h0005;
802   registers[6] = 16'h0088;
803   registers[7] = 16'h0007;
804 end
805
806 // Read operations
```

bus_8_data xxxx
ALU second source data (what will be in the alu) 0000
write_back_data 00xx
WB_mem_sel 2
write back enable 0
end

Time : 495
R0 : 0, R1 : 1, R2 : 256, R3 : 392, R4 : 4, R5 : 5, R6 : 136, R7 : -120

memory unit read address x data_out x , Time : 495

→ WAVEFORM:



Test 7:

In this test we wrote a code that tests the functionality of S-Type instructions

→ CODE:

```
147 sv r2, 0x6
148 lw r4, r1, 0x1
```

→ RESULT:

The screenshot shows the EDA Playground interface. On the left, there's a sidebar with 'Languages & Libraries' and 'Tools & Simulators'. The main area displays two Verilog files: 'testbench.sv' and 'design.sv'. The 'testbench.sv' file contains a module 'full_cpu_tb' that initializes a clock and a data output wire. The 'design.sv' file contains a register array 'registers' of 8 16-bit registers, initialized with values from 0x0000 to 0x0007. Below the code, there's a 'Log' section showing the simulation results, including a table of register values (R0 to R7) and a memory unit read address and data output.

```
module full_cpu_tb;
    initial begin
        $dumpFile("dump.vcd");
        $dumpvars;
    end
end

reg reset;
reg clk;
wire [15:0] data_out;

initial begin
    clk = 0;
    forever #5 clk = ~clk;
end
```

```
// Register array, 7 registers of 16 bits each
reg signed [15:0] registers [0:7];

// Initialize r0 to 0
initial begin
    registers[0] = 16'h0000;
    registers[1] = 16'h0001;
    registers[2] = 16'h0002;
    registers[3] = 16'h0003;
    registers[4] = 16'h0004;
    registers[5] = 16'h0005;
    registers[6] = 16'h0006;
    registers[7] = 16'h0007;
end

// Read operations
```

Time :	0	1	2	3	4	5	6	7
R0 :	0	1	R2 :	2	R3 :	3	R4 :	4

memory unit read address x data_out x , Time : 495

FETCH Time 495

→ WAVEFORM:

